

BENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Boxcars: Behaviour Oriented Concurrency in Rust

Author:
Alona Enraght-Moony

Supervisor:
Dr. Marios Kogias

Second Marker:
Dr. Lluís Vilanova

2024-06-17

Abstract

The **Rust** programming language offers extensive support for concurrent and parallel programming. While Rust is shown to be memory safe and data-race free, it is not deadlock free.

Behaviour-Oriented Concurrency is a novel paradigm that extends the actor model to allow atomically sending messages to multiple actors. Behaviour-Oriented Concurrency is both data-race-free and deadlock-free. **verona-rt** is a C++ library that provides an efficient implementation of a runtime for behaviour-oriented concurrency.

This project introduces the **boxcars** library, which implements behaviour-oriented concurrency in Rust. It provides an idiomatic and type-safe wrapper over the runtime from `verona-rt`.

The `boxcars` API enforces the guarantees of behaviour-oriented concurrency in ways that `verona-rt` is unable to. Using `boxcars` (instead of Rust's included `std::sync` library) also allows writing concurrent Rust code that is deadlock-free by construction.

Furthermore, we demonstrate that it imposes minimal overhead over using `verona-rt` directly.

Acknowledgements

First and foremost, I must thank Marios Kogias for his excellent job as project supervisor. His advice and guidance throughout the project has been stellar.

I'd also like to thank Matthew Parkinson, David Chisnall, and Sylvan Clebsch for providing critical feedback when presented with a much earlier version of the design, and pointing out a different direction to explore that ended up being much more fruitful than my initial attempts.

I'd like to thank Nora for pointing out to me how to make ThreadSanitizer work with Rust's standard library, and Mara Bos for chiming in at the ideal time to point out a subtle interaction with thread-local destructors.

Finally, I must give sincere thanks to Ed Page for developing and then introducing me to the [typos](#) tool, and to Nathaniel B for spotting about one million grammatical mistakes and typos on an earlier draft of this report. All remaining errors are mine.

Contents

1	Background	3
1.1	Rust	3
1.1.1	Ownership & Borrowing	3
1.1.2	Deadlocks	5
1.2	Behaviour-Oriented Concurrency	6
1.3	Verona Runtime	6
1.3.1	verona::cpp API	7
1.3.2	veronna::rt API	8
2	Design and Implementation of a Rust Library for Behaviour-Oriented Concurrency	11
2.1	Cowns	11
2.1.1	Maintaining the Reference Count	12
2.1.2	Construction and Destruction	13
2.2	Behaviours	14
2.2.1	On-Heap Layout	14
2.2.2	Invocation Trampoline	15
2.2.3	User-Facing Type-Safe API	16
3	Performance Evaluation	17
3.1	Microbenchmarks	17
3.1.1	Creating Cowns	17
3.1.2	Scheduling behaviours	18
3.2	Fibonarchi	18
4	Evaluation of User Experience	22
4.1	Splitting Borrows of Fields Behind a Cown	22
4.2	Acquiring the Same Cown Twice	24
4.3	Compiler Errors	25
4.3.1	Wrong Cown Constructor Arguments	25
4.3.2	Wrong When Lambda Argument Type	27
4.3.3	Not Using a Tuple for When Lambda Arguments	28
4.3.4	Passing Wrong Arguments as Cowns	28
4.4	auto-copy vs explicit clone	29
5	Future Works	32
5.1	Extending BoC and Boxcars	32
5.2	Passing Cowns between Rust and C++	32
5.3	Developing medium-scale software in BoC	32
	References	33

Chapter 1

Background

1.1 Rust

Rust (1) is a systems programming language.

It has type safety, memory safety and freedom from data-races.

Rust’s most important feature (for our purposes) is its system of **Ownership & Borrowing**.

1.1.1 Ownership & Borrowing

While a full tutorial on ownership and borrowing (and Rust more broadly) is well beyond the scope of this report, it is important to understand the principles at work, as they inform much of the later design. The core idea of **ownership** is that each value has a unique owner, and that value is dropped¹ when the owner goes out of scope. To quote The Book (1):

- Each value in Rust has an owner.
- [For each value] there can only be one owner at a time.
- When the owner goes out of scope, the value will be dropped.

Using just these rules, Rust could offer a type-safety and memory-safety. However, it would be extremely cumbersome to program in. For example, when a function took a parameter, that function becomes the owner of that parameter, and the value can no longer be used.

```
let x: String = make_string(); // `x` owns a `String`
do_thing_with_string(x);      // ownership of `x` transferred
do_other_thing_with_string(x); // `x` cannot be used here
```

Listing 1.1: A demonstration of ownership

For example, in listing 1.1, the variable `x` is the owner of a value of type `String`. However, when calling `do_thing_with_string`, ownership is moved (or *transferred*) to that function. This means that the value cannot be used in the call to `do_other_thing_with_string`. Indeed, this is what the compiler error in listing 1.2 shows us.

This is different to C++, where objects can be used after they are `std::moved` from. However, the standard places no guarantees on the content of those objects, only saying “moved-from objects shall be placed in a valid but unspecified state” (2).

To avoid this constantly becoming a footgun for users, C++ values won’t be moved by default, but instead will be copied, which leaves the original value intact. This is shown by 1.3, where `x` is used both values are copied by default.

¹“dropped” here refers to running the associated cleanup action for a given value, such as freeing an allocation, or closing a file. It’s broadly equivalent to C++’s destructors.

```

error[E0382]: use of moved value: `x`
  --> src/main.rs:4:32
   |
2  | let x: String = make_string();
   |     - move occurs because `x` has type `String`, which does not implement the
   |     ↪ `Copy` trait
3  | do_thing_with_string(x);
   |     - value moved here
4  | do_other_thing_with_string(x);
   |                               ^ value used here after move

```

Listing 1.2: Compiler error from listing 1.1

```

std::string x = make_string();
do_thing_with_string(x);           // `x` copied into `do_thing_with_string`
do_other_thing_with_string(x);    // `x` still has it's contents here and can be used

```

Listing 1.3: Demonstration of copies in C++

If you want to move a value (and not pay the cost of creating a new value), C++ makes you do this explicitly, by calling the `std::move` function, as shown in listing 1.4. However, unlike the Rust equivalent (listing 1.1) this does compile, but `do_other_thing_with_string` may see any value, as it's using `x` after it's been moved from ².

```

std::string x = make_string();
do_thing_with_string(std::move(x)); // `x` copied into `do_thing_with_string`
do_other_thing_with_string(x);     // `x` "valid but unspecified" here.

```

Listing 1.4: Demonstration of moves in C++

In order to support this, C++ has a concept of copy and move constructors ³, which allow users to run custom code whenever a value is moved (e.g. to set the moved-from allocation to NULL) or copied (e.g. to create a new allocation). In contrast, Rust moves are always bitwise (ie the result of memcpying the value from the old to new location). However, as the old value is guaranteed to never be accessed, libraries don't need to modify it on the way out, to avoid use-after-free.

1.1.1.1 Borrowing

If values could only be owned and moved, than programming in Rust would be extremely unergonomic. As shown, all values could only be used by a function once, and then would no longer be accessible ⁴.

Therefore ⁵, Rust has the additional notion of **borrowing**. A value can be borrowed in one of two ways: by an immutable reference (spelt `&T`), or an mutable reference (`&mut T`).

²In practice, this will be the empty string, but well written programs should not rely on this behaviour.

³This means to fully support managing ownership of a value, a C++ type should overload:

1. Its destructor
2. Its copy constructor
3. Its copy assignment operator
4. Its move constructor
5. Its move assignment operator

This is known as the "Rule of 5" (3).

⁴This could be somewhat circumvented by having a function return back its arguments to the caller, but this would be extremely cumbersome.

⁵And for other reasons as well.

A value may have many shared references to it at a given time, but if it has any exclusive reference to it that reference must be the only one. With a shared reference, you can only read from the value. An exclusive reference is required to mutate it. More succinctly, Rust references are “Aliasable XOR mutable” (5). The fact that an immutable reference can be shared, while a mutable reference must be exclusive has lead to these sometimes being call shared references (for &T) and exclusive references (for &mut T) (6).

Borrowed values have a **lifetime** for which they are borrowed. This is needed to ensure that all exclusive references don’t overlap with shared ones. This is enforced by a part of the compiler called the “borrow checker”, that assigns a lifetime to each borrow, and uses this to determine if there are ever any aliased mutable borrows. Listing 1.5 contains an example of the lifetimes assigned to various borrows.

```
let x: i32 = 0;

let shared_1: &i32 = &x; // Lifetime 1 starts
dbg!(shared_1);
// Lifetime 1 stops.

let exclusive_2: &mut i32 = &mut x; // Lifetime 2 starts

// It would be a compiler error to borrow `x` here (in either way), as it's already
// borrowed mutably/exclusivly by `exclusive_2`

*exclusive_2 += 10;
// Lifetime 2 ends

// But now that lifetime 2 is over, we can borrow `x` again

let shared_3: &i32 = &x; // Lifetime 3 starts
let shared_4: &i32 = &x; // Lifetime 4 starts

dbg!(shared_4);
// Lifetime 4 ends
dbg!(shared_3);
// Lifetime 3 ends
```

Listing 1.5: Demonstration of lifetimes

Because the borrow checker knows what values each borrow is borrowing from, it is able to ensure that borrows are not dangling (instead of doing UB like it would in C++). Listing 1.6 attempts to use a borrow to `short_lived` after it’s gone out of scope (and therefore dropped). This is caught by the compiler, as shown be the error in listing 1.7, whereas in C++ it would be undefined behaviour.

```
let mut x: &i32 = 0;
{
    let short_lived: i32 = 0;
    x = &short_lived;
} // `short_lived` goes out of scope here.
dbg!(x);
```

Listing 1.6: Attempting to use a reference to a value that’s out of scope

1.1.2 Deadlocks

However, one area where Rust’s type system doesn’t do anything to help you is avoiding deadlocks. One can trivially perform one by acquiring two mutexes in different orders, as shown in listings 1.8 and 1.9.

```

error[E0597]: `short_lived` does not live long enough
--> src/main.rs:5:13
4 |         let short_lived: i32 = 0;
  |         ----- binding `short_lived` declared here
5 |         x = &short_lived;
  |         ^^^^^^^^^^^^^^^ borrowed value does not live long enough
6 |     }
  |     - `short_lived` dropped here while still borrowed
7 |     dbg!(x);
  |     - borrow later used here

```

Listing 1.7: Compiler error from listing 1.6

While this example is obviously contrived, it demonstrates an important point, which that while Rust can statically prevent data-races Rust deadlocks have come up in practice, and are usually much more subtle than this (7,8). This is such a problem that `parking_lot`, an library with alternative mutex implementation to the standard library, offers an optional runtime deadlock-detector (9).

1.2 Behaviour-Oriented Concurrency

Behaviour-Oriented Concurrency (BoC) is a novel concurrency paradigm (10). It has 2 key components:

- **Cowns:** A cown (short for concurrent owner) is a piece of data.

A cown can be in one of two states: available or acquired. An available cown is eligible to be acquired by a behaviour, but its associated data cannot be accessed. An acquired cown can have its data accessed, but only while it's acquired.

The only way to acquire a cown (and thus access its data) is to spawn a behaviour onto it, and cowns are only acquired for the duration of that behaviour, after which they become available again.

- **Behaviours:** A behaviour is a unit of execution that acts upon a set of cowns.

When a behaviour is spawned, it is given the set of cowns it will acts on, as well as the code to run on them. When all the required cowns can be acquired, the code is executed, and then the cowns are made available again.

A cown can only be acquired by one behaviour at any given. This can be thought of as being like each cown having a mutex, which is locked before the behaviour starts and unlocked after it ends. However, every cown in a behaviour is acquired atomically, and there is no change for deadlock.

Note that this means that spawning a behaviour returns immediately, and the code inside will be executed at some indetermined future point, when unique access to all cowns can be guaranteed.

Listing 1.10 contains a simple example of BoC code in some hypothetical language.

Inside the block for the `when`, a user can access the data inside a cown. However, outside then `when`, when that cown hasn't been acquired, it's invalid to access it.

BoC is both *data-race free* and *deadlock free*. No data races can occur, as cowns can only be modified when acquired by behaviours. No deadlocks can occur, as spawning a behaviour doesn't block but insteads schedules the behaviour to be executed when all its cowns can be acquired.

1.3 Verona Runtime

The Verona Runtime (also known as `verona-rt`) is a C++ library that implements behaviour oriented concurrency. It is intended to be a part of the currently in development Verona language, but it can also be used as a freestanding C++ library today (10).

```
fn main() {
    let m1 = Mutex::new(());
    let m2 = Mutex::new(());

    scope(|s| {
        s.spawn(|| {
            let g1 = m1.lock();
            sleep_ms(100); // not strictly necessary, but makes it more likely.
            println!("t1: got m1, trying to get m2");
            let g2 = m2.lock();
            println!("t1: got both");
        });

        s.spawn(|| {
            let g2 = m2.lock();
            sleep_ms(100);
            println!("t2: got m2, trying to get m1");
            let g1 = m1.lock();
            println!("t2: got both");
        });
    });
}
```

Listing 1.8: Rust code that deadlocks by acquiring mutexes in different orders

```
t1: got m1, trying to get m2
t2: got m2, trying to get m1
```

Listing 1.9: Stdout from executing listing 1.8

It exposes both a the `verona::cpp` API, which uses templates to expose a typed C++ api for cowns and behaviours, as well as the lower level `verona::rt` API, which is used to implement `verona::cpp`, but can also be used in it's own right.

1.3.1 `verona::cpp` API

The `verona::cpp` API lets users write BoC programs in a nice DSL. For cowns, it exposes the `cown_ptr<T>` type, which holds the underlying `T` behind a reference-count, but doesn't allow access to it directly. This enforces the invariant in BoC that cowns can only be accessed when they're acquired.

To acquire a cown, you can use the `when` function. This takes in the cowns you want to acquire, and a lambda to invoke on those cowns. This lambda is passed a parameter of type `acquired_cown<T>`. This represents the same cown that was passed in the argument, but with the idea that it's acquired (and therefore allowed to mutate the cown's underlying data) encoded in the type system. Listing 1.11 shows an example of this ⁶.

Unlike `cown_ptr<T>`, `acquired_cown<T>` does allow access to and mutation of the underling data. By overloading `operator->`, `operator*` and `operator T&`, it is able to provide transparent access to the underlying data, as shown in listing 1.12.

But when a cown isn't acquired, as in listing 1.13, it's a compiler error to attempt to access the contained data (listing 1.14).

⁶I'm choosing to give the type declaration of all variables for clarity. On actual code, most of these can be inferred, and will be written as `auto` instead.

```

var myCown: Cown[int] = cown.create(10);

when(myCown) {
    myCown += 10;
}

myCown += 10; // invalid.

```

Listing 1.10: Demonstration of BoC

```

cown_ptr<int> my_cown = make_cown<int>(10);
when(my_cown) << [] (acquired_cown<int> my_cown) {
    // cown acquired in here.
};

```

Listing 1.11: Creating and acquiring a cown with the `verona::cpp` library

1.3.1.1 How `cown_ptr` and `acquired_cown` model BoC

In order to have `cown_ptr` and `acquired_cown` correctly model the semantics of a cown in BoC, they must do a few things.

1. A `cown_ptr` must always point to a non-dangling cown.
2. An `acquired_cown` must only be accessible when that cown has been acquired.

The first of these is achieved by having `cown_ptr` overload its constructors, assignment operators, and destructor to maintain the cown's internal reference-count. Much like `std::shared_ptr`, this means that when the user does `cown_ptr<int> b = a`, the reference count of the cown is incremented.

To achieve the second, `acquired_cown` is made to only be constructable by certain classes inside the `verona::cpp` namespace. These will only do so in order to pass to the lambda given to `when`, and in no other circumstance. Therefore, the only way to obtain an `acquired_cown` is when it has been acquired by a behaviour created with `when`.

In addition `acquired_cown` detete's its move and copy constructors, and its assignment operators, so that users cannot store it anywhere other than the parameter of the lambda. This means that attempts to store an `acquired_cown` such that it can be accessed later (e.g. listing 1.15) are turned into compiler error (e.g. listing 1.16).

1.3.1.2 Circumventing access guarantees via pointer.

While the behaviour discussed above (§1.3.1.1) ensures that an `acquired_cown` can only be accessed when that cown is acquired, this doesn't ensure that the *cown's data* can only be accessed when that cown is acquired. As shown in listing 1.17, a user can get a pointer to the underlying data from an `acquired_cown`, and nothing prevents that pointer being used to access the cown even after it's no longer acquired by the behaviour in which it was obtained.

Because users can access a cown's data when it is no longer acquired, the `verona::cpp` implementation of BoC fails to provide data-race freedom, even though the underlying BoC model it does. This is because C++ isn't expressive enough to encode the idea that an acquired cown (the BoC concept, not the C++ implementation of said concept) can only be accessed for a limited duration. This is because C++ has no mechanism to limit how long a pointer/reference can be used, and the `acquired_cown` class must give out pointers/references to allow access to the underlying data.

Rust can solve this with the power of lifetimes.

1.3.2 `verona::rt` API

`verona::rt` is a lower-level API that is used to implement the `verona::cpp` API. While it still has `Cowns`, they aren't typed at all. It instead just exposes the `verona::rt::Cown` type. Higher level APIs can use its implementation of reference-counting and scheduling, but they themselves are responsible for calling `verona::rt::Cown`'s methods to change the reference-count, and schedule.

```

auto f = make_cown<std::string>("hello");

when(f) << [](auto f) {
    f->push_back('!');
    std::cout << *f << '\n';
};

```

Listing 1.12: Mutating and accessing a cown that has been acquired

```

auto f = make_cown<std::string>("hello");

when(f) << [](auto f) {
    f->push_back('!'); // This is ok, cown acquired
};

f->push_back('?'); // Compiler error, cown not acquired

```

Listing 1.13: Attempting to access a non-acquired cown

Additionally, `verona::rt::Cown` doesn't encode any information about the data it contains. Higher level users are responsible for placing this alongside the `verona::rt::Cown` (ideally in the same allocation), and then making this data only accessible inside behaviours that have acquired that cown.

```

playground.cc: In function ‘void real_main()’:
playground.cc:35:4: error: base operand of ‘->’ has non-pointer type
  ↪ ‘verona::cpp::cown_ptr<std::__cxx11::basic_string<char> >’
   35 |     f->push_back('?');
      |         ^~

```

Listing 1.14: Compiler error from listing 1.13

```

cown_ptr<int> cown = make_cown<int>(10);
when(cown) << [](acquired_cown<int> f) { squirrel_away_for_later(f); };

```

Listing 1.15: Attempting to store a acquired_cown for use later

```

playground.cc: In lambda function:
playground.cc:35:67: error: use of deleted function
  ↪ ‘verona::cpp::acquired_cown<T>::acquired_cown(const
  ↪ verona::cpp::acquired_cown<T>&) [with T = int]’
   35 |     when(cown) << [](acquired_cown<int> f) { squirrel_away_for_later(f); };
      |                                     ~~~~~^~~~~
In file included from verona-rt/src/rt/./cpp/when.h:6,
                  from playground.cc:1:
verona-rt/src/rt/./cpp/cown.h:464:5: note: declared here
   464 |     acquired_cown(const acquired_cown&) = delete;
      |     ^~~~~~
playground.cc:27:49: note:   initializing argument 1 of ‘void
  ↪ squirrel_away_for_later(verona::cpp::acquired_cown<int>)’
   27 | void squirrel_away_for_later(acquired_cown<int> acq);
      |                               ~~~~~^~~~~

```

Listing 1.16: Compiler error from listing 1.15

```

cown_ptr<int> my_cown = make_cown<int>(10);
int* escape_data;
when(my_cown) << [&escape_data](acquired_cown<int> my_cown) {
    escape_data = &*my_cown;
};
sleep(3); // Wait for behaviour to run
*escape_data += 10; // Modifies cown without acquiring!!!

```

Listing 1.17: Accessing a cowns data when that cown isn’t acquired

Chapter 2

Design and Implementation of a Rust Library for Behaviour-Oriented Concurrency

The `boxcars` library implements BoC cowns and behaviours on top of the `verona-rt` runtime.

```
let c1 = Cown::new("hello");
let c2 = Cown::new("world");

let n = 10;

when((&c1, &c2), |(c1, c2)| {
    println!("{c1} {c2}!");
    println!("n = {}", n);
});
```

Listing 2.1: A simple example of using boxcars

It has 3 main parts to its public API.

- `Cown`, which represents an unacquired cown
- `when`, which schedules a lambda onto a collection of cowns.
- `AcquiredCown`, which is passed to the lambda, and gives access to the cowns data.

A significant challenge is that Rust code can't call C++ functions directly, only C ones.

2.1 Cowns

The first thing we need to do is have a `Cown` type that represents some data. The cown will be a pointer to a heap allocation that contains:

1. Scheduling state
2. Reference count
3. The data.

1 and 2 should be managed by the `verona-rt` library, but it's unable to know about the user data. That's because the data is a generic parameter, and this won't work over FFI.

The rust `Cown<T>` type is a wrapper over a `verona::rt::Cown*`. Note that the rust-side type carries type information, but the C++ side doesn't. That's because `T` can be any Rust type. The full declaration is given in listing 2.2

1.

```
pub struct Cown<T> {
    cown_ptr: CownPtr,
}

/// a `verona::rt::Cown*`
#[repr(transparent)]
struct CownPtr {
    addr: *mut (),
}
```

Listing 2.2: Declaration of `boxcars::Cown` type

`CownPtr` is kept as its own type, as it's useful for both `boxcars::Cown` and `boxcars::AcquiredCown`. It must be marked as `#[repr(transparent)]` to ensure it is treated like a pointer at an ABI level (11).

2.1.1 Maintaining the Reference Count

As `Cown<T>` is a reference-counted pointer to some on-heap `T`, we need to ensure the reference-count is maintained. Unlike `verona::cpp::cown_ptr` (§1.3.1.1), we cannot override constructors or assignment operators, as Rust only supports bitwise moves (§1.1.1). This is different to C++, due to Rust's different ownership model. Rust will never implicitly call user code when assigning an object, but as the moved-from value isn't dropped, no reference-count updates are needed in these cases.

However, we still want a way to duplicate a `Cown` and increment the reference count. For this sort of action², Rust uses the `Clone` trait, which is used in broadly the same situations as a copy constructor in C++³, but requires being called explicitly. Similarly, we can implement the `Drop` trait to decrement the reference count when the value goes out of scope. These implementations, shown in listing 2.3 and 2.4, simply call into foreign function interface (FFI) code that can manipulate the reference count.

```
impl<T> std::clone::Clone for Cown<T> {
    fn clone(&self) -> Self {
        unsafe {
            ffi::boxcars_acquire_object(self.cown_ptr);
        }

        Self {
            cown_ptr: self.cown_ptr,
            _marker: PhantomData,
        }
    }
}
```

Listing 2.3: `Clone` implementation for `boxcars::Cown`

Then on the C++ side, we have very simple helper functions, that forward to `verona-rt`'s reference-counting implementation (listing 2.5). We can't call the `verona::rt::Cown::acquire` static method directly from Rust, as

¹The actual declaration of `Cown` also has a `PhantomData` field, which is needed as Rust disallows unused type parameters. However it's elided here, for not being relevant.

²Duplicating a value.

³Confusingly, what C++ calls "copyable" (`std::copyable`) is equivalent to Rust's idea of "cloneable" (`std::clone::Clone`), both of which broadly mean an object can be duplicated by potentially running code. Meanwhile what Rust calls "copyable" (`std::marker::Copy`) instead corresponds to C++'s idea of "trivially copyable" (`std::is_trivially_copyable`), both of which correspond to types that can be duplicated purely by duplicating the underlying bytes.

This has caused endless confusion, as the difference between these two concepts is super important, and the two languages with the most support for this kind of thing choosing to use "copy" to mean the two different things is extremely unfortunate.

Speaking personally, I learn the Rust terminology far before the C++ one, and think in terms of it.

```
impl<T> std::ops::Drop for Cown<T> {
    fn drop(&mut self) {
        unsafe { ffi::boxcars_release_object(self.cown_ptr) };
    }
}
```

Listing 2.4: Drop implementation for `boxcars::Cown`

it's not declared as `extern "C"`, so it will have a mangled name that Rust code won't be able to link to⁴. Importantly, despite the `Clone` and `Drop` implementation being generic over `T`, the `boxcars_acquire_object` and `boxcars_release_object` functions aren't generic/templated. This is because `boxcars` maintains a strict separation, where the C++ side never knows about the Rust data being stored in cowns, and the Rust side doesn't know how the C++ represents the scheduling/reference-counting state of cowns.

```
using verona::rt::Cown;

extern "C"
{
    void boxcars_acquire_object(Cown* o)
    {
        Cown::acquire(o);
    }
    void boxcars_release_object(Cown* o)
    {
        auto& alloc = verona::rt::ThreadAlloc::get();
        Cown::release(alloc, o);
    }
}
```

Listing 2.5: C++ side of reference counting

2.1.2 Construction and Destruction

The remaining operations are creating a new `Cown`, and running the destructor on the underlying data once the reference-count reaches zero. It turns out these are closely related.

The core problem when creating a `Cown` was that the `verona::rt::Cown` could only be created from C++, but C++ couldn't know about the `T` in a `boxcars::Cown<T>`. However, we need to know what `T` in order to create an appropriately sized heap allocation. But this is the only property that's required, and we can communicate this via a simple parameter of type `size_t` (instead of needing to template). The C++ `size` will do a heap allocation of the requested size, and initialise a `verona::rt::Cown` at the top of it. It can then pass a pointer to this allocation back to Rust, which moves its `T` to the bottom.

This is implemented in listing 2.6. It creates a descriptor of the cown it would like (which includes the size), and then passes that over to C++. The C++ only initialises the `verona::rt::Cown`, Then the Rust moves value into that heap allocation using `std::ptr::write`.

Figure 2.1 shows what this looks like in memory. The single heap allocation for a cown contains both the `verona::rt::Cown`, which stores scheduling and reference counting information, as well as the user's data. It also has a pointer to a (per-type) descriptor. This descriptor stores a pointer to the destructor function, which is called when the reference count reaches 0.

The rust `get_descriptor` function returns a pointer to a statically-allocated descriptor. It contains the size needed

⁴By default, both Rust and C++ mangle their symbol names when producing object files. To make FFI calls between them, both languages must use `extern "C"` to tell the compiler not to use name mangling, and to use the C ABI/calling convention.

```

impl<T> Cown<T> {
    pub fn new(value: T) -> Self {
        unsafe {
            let desc = get_descriptor::<CownData<T>>();
            let cown_ptr = ffi::boxcars_allocate_cown(desc);
            let this = Self { cown_ptr };
            std::ptr::write(this.data_ptr(), value);
            this
        }
    }
}

```

Listing 2.6: Implementation of `boxcars::Cown::new`

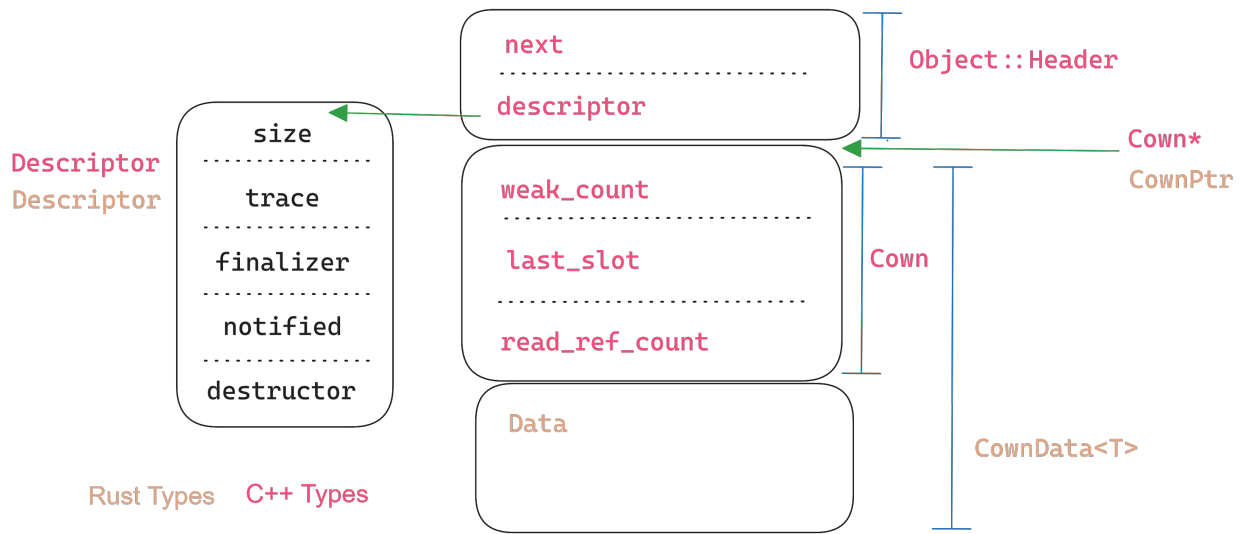


Figure 2.1: In-memory layout of a cown in boxcars

for the underlying heap allocation, as well as a pointer to the type-erased destructor that will run the destructor for the Rust-managed data after the reference-count reaches 0, but before the cown’s allocation is freed.

2.2 Behaviours

2.2.1 On-Heap Layout

A behaviour is represented as a single heap allocation, as shown in figure 2.2. This contains many things.

1. `verona::rt::Work`. This contains some scheduling state, as well as a function pointer (`void (*f)(Work*)`) to be invoked when the behaviour is scheduled.
2. `verona::rt::BehaviourCore`. This contains more scheduling state ⁵, as well as the number of slots.
3. Array of `verona::rt::Slot/boxcars::Slot`. These contain the Cowns that the behaviour must acquire before being executed. Unlike `Work` and `BehaviourCore`, the Rust side of `boxcars` knows the layout of these structs, so it can read the cowns from them.
4. `impl FnOnce(&[Slot])`. This is called the “payload”, and is completely opaque to `verona-rt`. It is read by the function pointer inside the `Work`, and used to store the closure captures.

To create a behaviour, the low-level function is `boxcars_sched_lambda`, whose type signature is given in listing 2.7. It creates a behaviour structure (shown in figure 2.2), that contains `n_cowns` cowns copied from `cowns` ⁶,

⁵This begs the question: Why are `Work` and `BehaviourCore` separate? `verona-rt` wants to explore scheduling things that aren’t behaviours, and uses this as a common abstraction.

⁶We can’t use a slice of cowns (`&[CownPtr]`) here, because Rust slices don’t have a well-defined ABI. Instead we decompose it into its raw parts (a pointer and a length), and pass those individually.

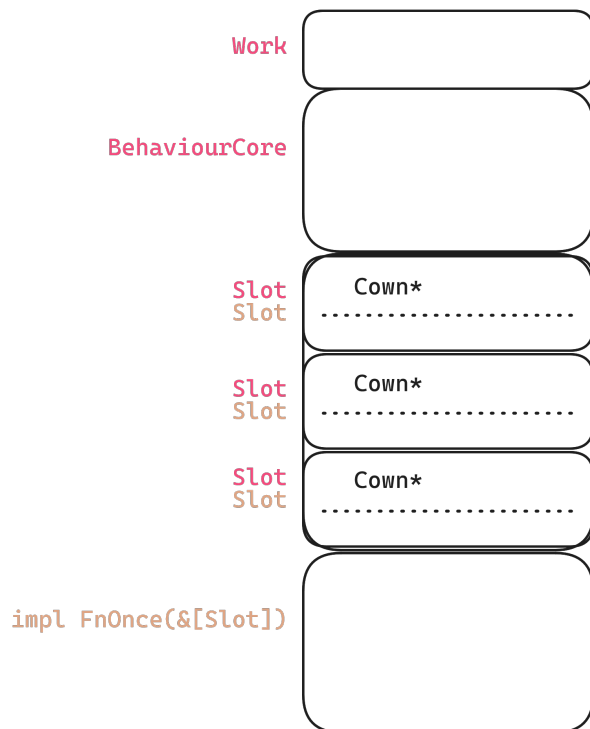


Figure 2.2: In-memory layout of a behaviour in boxcars

payload_size bytes of payload copied from payload, and f as the pointer that will be invoked when all cowns are acquired.

```
extern "C" {
    fn boxcars_sched_lambda(
        f: extern "C" fn(WorkPtr),

        cowns: *const CownPtr,
        n_cowns: usize,

        payload: *const (),
        payload_size: usize,
    );
}
```

Listing 2.7: Rust-side declaration of boxcars_sched_lambda

2.2.2 Invocation Trampoline

The function passed as f here can't be the users closure they passed to when, as that expects to receive AcquiredCowns. Instead, always use the invoke_trampoline (listing 2.8) function. It's instantiated with a different F for each different closure. How it works is

```
extern "C" fn invoke_trampoline<F>(work: WorkPtr)
where
    F: FnOnce(&[Slot])
```

Listing 2.8: Signature of invoke_trampoline

1. Taking the pointer to `Work` (as the other data stored after it, see figure 2.2), as using that to find

- The pointer to the start of the slots array
- The length of the slots array
- The pointer to the start of the payload, where the closure's captures are stored.

2. Invoke the closure.

Importantly, the closure isn't called via a function pointer. The payload pointer is only used to pass the closure's captures, but the definition itself is passed via the generic argument `F`, and therefore the call is direct (and likely to be inlined) due to monomorphization.

3. Free associated resources. This involves:

- Releasing the acquired cows, so they can be acquired by the next behaviour that's scheduled onto them
- Running the destructor on the closure's captures (if applicable).
- Freeing the heap allocation that contains the behaviour.

2.2.3 User-Facing Type-Safe API

Chapter 3

Performance Evaluation

One important metric to evaluate the project on is performance. As discussed previously (§2.1), it's not simple to call into the C++ runtime from Rust, and I had to be somewhat indirect due to the FFI boundary. I wanted to measure the performance overhead of this, versus C++ code that can call it directly.

3.1 Microbenchmarks

3.1.1 Creating Cowns

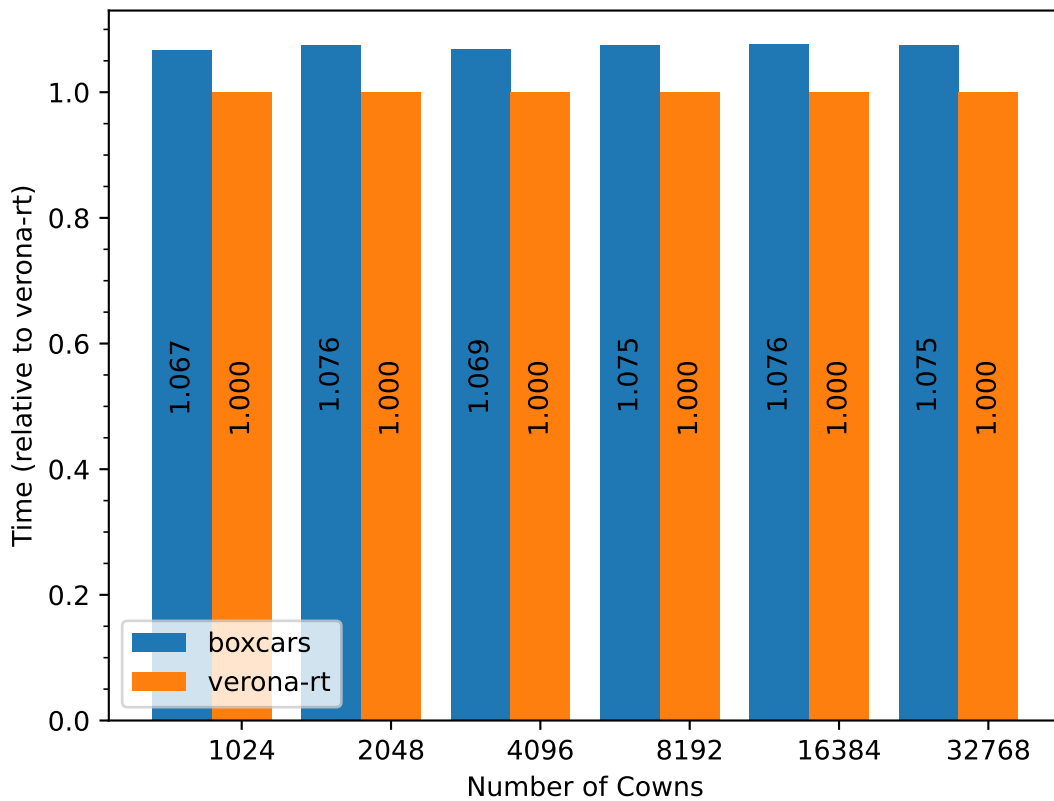


Figure 3.1: Time to create cowns

The first benchmark I wrote was to create a vector of n Cowns, and then free them. This was to measure the overhead of Rust not being able to interact directly with the Cown constructors, but having to do it via FFI, as discussed previously (§2.1).

As figure 3.1 shows, there's a roughly 7% overhead when `boxcars` creates a `cown`, vs when `verona-rt` does it.

```
let mut v = Vec::with_capacity(n);
for i in 0..n {
    v.push(Cown::<usize>::new(i));
}
```

Listing 3.1: `boxcars` implementation of figure 3.1

```
std::vector<cown_ptr<size_t>> v;
v.reserve(n);

for (size_t i = 0; i < n; i++)
{
    v.push_back(make_cown<size_t>(i));
}
```

Listing 3.2: `verona-rt` implementation of figure 3.1

3.1.2 Scheduling behaviours

The other major implementation difference is how behaviours are scheduled onto the `Cowns`, with Rust also needing indirection over FFI (§2.2). Therefore I benchmarked scheduling a large number of behaviours that do minimal work (listings 3.3 and 3.4), to measure the cost of the scheduling and execution itself.

As shown in figure 3.2, `boxcars` imposes very-little overhead over `verona-rt` for scheduling behaviours.

```
let c = Cown::new(0);
for _ in 0..n {
    when(&c, |mut c| {
        *c += 1;
    })
}
```

Listing 3.3: `boxcars` implementation of figure 3.2

3.2 Fibonarchi

For a more involved benchmark, I tried a parallel evaluation of the `Fibonarchi` function. In my initial initial benchmarks, the `boxcars` code was performed much better. The difference was so great, I didn't think it could be caused by differences between `boxcars` and `verona-rt`, but instead I was somehow doing less work. This was indeed the case, as C++ will implicitly call the copy-constructor on `cown_ptr`, but Rust makes this explicit by calling `.clone()`. Because this extra reference-counting was made explicit, I'd avoided it in my initial Rust, but it happened in C++.

I ended up with 4 implementations:

1. **careful boxcars**: My original Rust implementation (listing 3.5).
2. **uncareful verona**: My original C++ implementation, that does unnecessary reference-counting (listing 3.6).
3. **careful verona**: A C++ implementation that avoids unnecessary reference-counting (listing 3.7).
4. **uncareful boxcars**: A Rust implementation that adds the unnecessary reference-counting from 2 (listing 3.8).

The performance of these is compared in figure 3.3. Both the careful implementations have almost exactly the same performance, while both the uncareful ones do significantly worse.

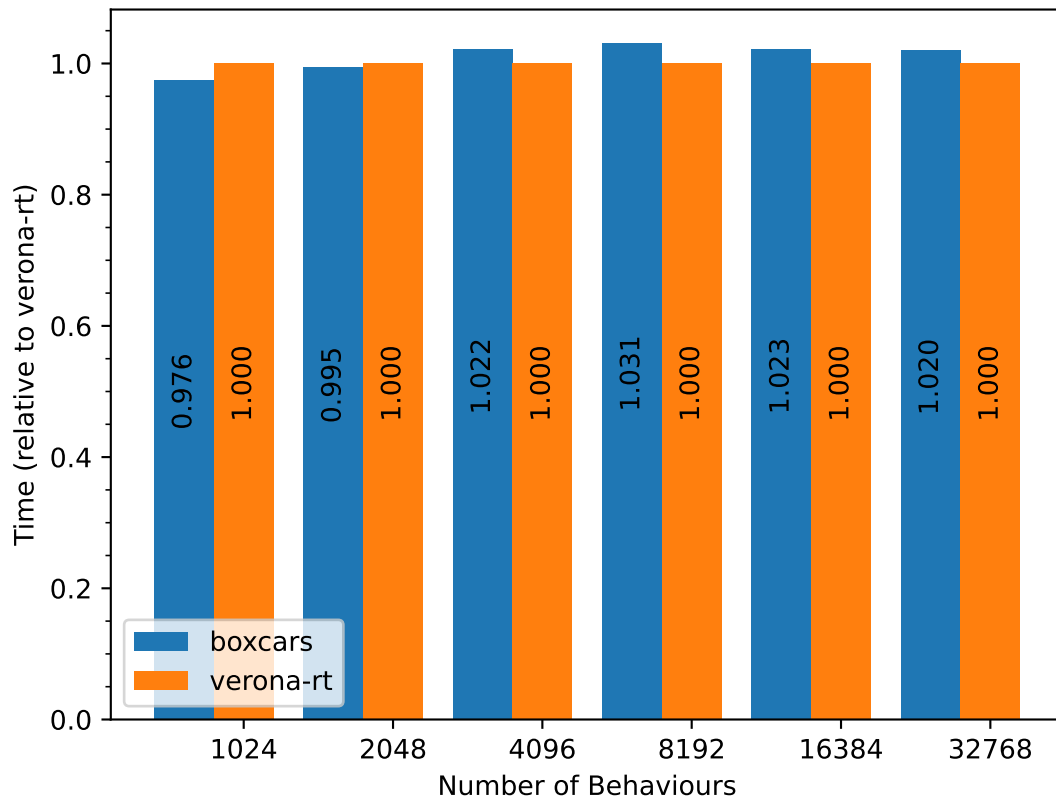


Figure 3.2: Time to schedule behaviours

```

auto c = make_cown<int>(0);
for (int j = 0; j < n; j++)
{
    when(c) << [](auto c) { c++; };
}

```

Listing 3.4: verona-rt implementation of figure 3.2

```

fn par_fib_careful(n: u32, result: &Cown<u32>) {
    if n <= 4 {
        when(result, move |mut r| *r = sequential_fib(n));
    } else {
        let f1 = Cown::new(0);
        par_fib_careful(n - 1, &f1);
        par_fib_careful(n - 2, result);
        when((result, &f1), |(mut r, f)| *r += *f);
    }
}

```

Listing 3.5: Implementation of ‘careful boxcars’ from figure 3.3

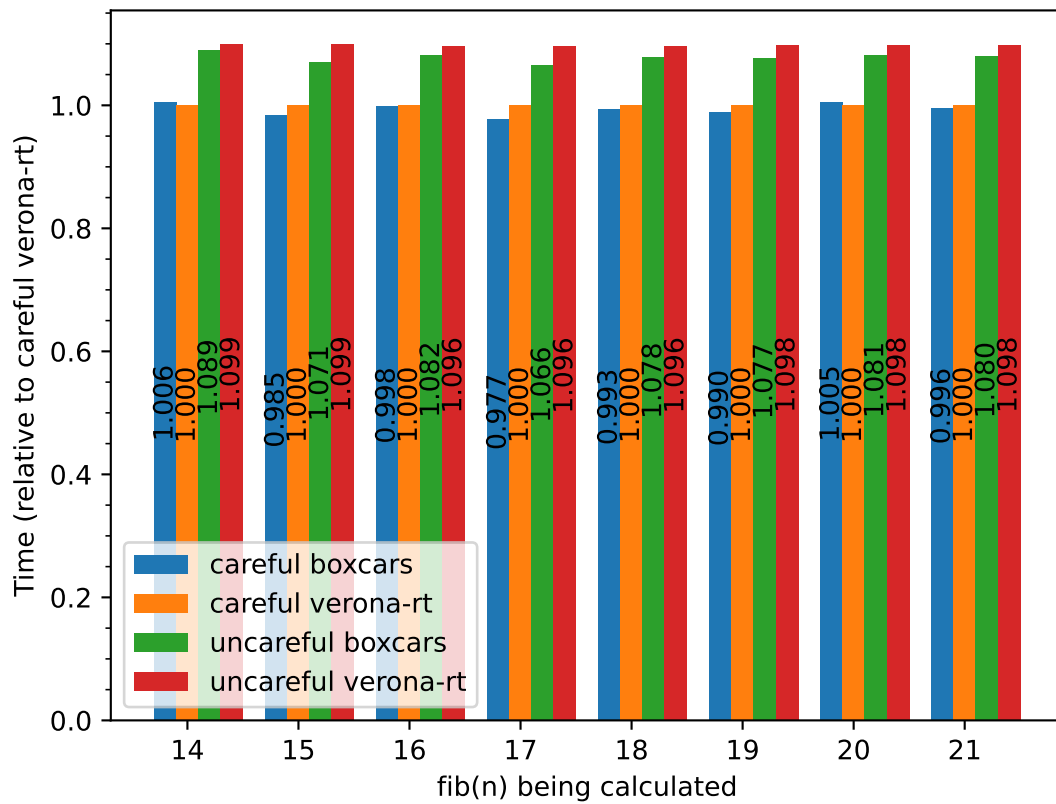


Figure 3.3: Time to calculate $fib(n)$

```

void par_fib_uncareful(uint32_t n, cown_ptr<uint32_t> result)
{
    if (n <= 4)
    {
        when(result) << [n](auto r) { *r = sequential_fib(n); };
    }
    else
    {
        auto f1 = make_cown<uint32_t>(0);
        par_fib_uncareful(n - 1, f1);
        par_fib_uncareful(n - 2, result);
        when(result, f1) << [] (auto r, auto f) { *r += f; };
    }
}

```

Listing 3.6: Implementation of ‘uncareful verona-rt’ from figure 3.3

```

void par_fib_careful(uint32_t n, cown_ptr<uint32_t>& result)
{
    if (n <= 4)
    {
        when(result) << [n](auto r) { *r = sequential_fib(n); };
    }
    else
    {
        auto f1 = make_cown<uint32_t>(0);
        par_fib_careful(n - 1, f1);
        par_fib_careful(n - 2, result);
        when(result, f1) << [](auto r, auto f) { *r += f; };
    }
}

```

Listing 3.7: Implementation of ‘careful verona-rt’ from figure 3.3

```

fn par_fib_uncareful(n: u32, result: Cown<u32>) {
    if n <= 4 {
        when(&result, move |mut r| *r = sequential_fib(n));
    } else {
        let f1 = Cown::new(0);
        par_fib_uncareful(n - 1, f1.clone());
        par_fib_uncareful(n - 2, result.clone());
        when((&result, &f1), |(mut r, f)| *r += *f);
    }
}

```

Listing 3.8: Implementation of ‘uncareful boxcars’ from figure 3.3

Chapter 4

Evaluation of User Experience

Of course, performance isn't the only thing worth measuring. It's also worth considering what it's like to *use* these libraries, and how the language and API differences impact what it's like to write BoC code in them.

4.1 Splitting Borrows of Fields Behind a Cown

While most of the time `AcquiredCown<'a, T>` acts like a transparent wrapper over `&'a mut T`, there are some cases where this abstraction becomes leaky, and users need to be aware that they're not dealing with a normal reference.

```
let cown = Cown::<i32>::new(10);

// This type annotation isn't needed, but makes the coercion clearer
//
when(&cown, |acq_cown: AcquiredCown<i32>| {
    let n: i32 = *acq_cown;
});
```

Listing 4.1: Demonstration of auto-deref

In code like that in listing 4.1, `acq_cown` acts like an `&mut i32`, and is able to be dereferenced into `i32`. This even extends to method calls, as shown in listing 4.2, where we can call `to_uppercase` (a method defined on `&str`) on an `AcquiredCown<&str>`.

```
let cown = Cown::<&str>::new("hello");

when(&cown, |acq_cown: AcquiredCown<&str>| {
    let uppercase_str = acq_cown.to_uppercase();
});
```

Listing 4.2: Calling methods on acquired cowns via auto-deref

This is possible because `AcquiredCown` implements the `Deref` and `DerefMut` traits. The Rust compiler will implicitly insert calls to these traits' methods to allow an `AcquiredCown<T>` to be treated like an `&mut T`. This normally works seamlessly, as shown in listings 4.1 and 4.2.

However, this abstraction doesn't always hold. The code in listing 4.3 compiles and executes successfully. Unfortunately a naïve translation of this code to BoC (listing 4.4) fails to compile, giving the error shown in 4.5.

```
struct Foo {
    a: i32,
    b: i32,
}

fn use_ints(x: &mut i32, y: &mut i32) {
    *x += 1;
    *y += 1;
}

let mut foo = Foo { a: 1, b: 1 };
use_ints(&mut foo.a, &mut foo.b);
```

Listing 4.3: Demonstration of partial borrowing

```
let cown = Cown::new(Foo { a: 1, b: 1 });
when(&cown, |mut acq_cown: AcquiredCown<Foo>| {
    use_ints(&mut acq_cown.a, &mut acq_cown.b)
});
```

Listing 4.4: Attempting to borrow two fields of a struct in a cown

```
error[E0499]: cannot borrow `acq_cown` as mutable more than once at a time
--> crates/verona-rt/examples/play.rs:6:40
    |
6  |   use_ints(&mut acq_cown.a, &mut acq_cown.b)
    |   -----          -----          ^^^^^^^^^ second mutable borrow occurs here
    |   |               |
    |   |               first mutable borrow occurs here
    |   first borrow later used by call
```

Listing 4.5: Compiler error from listing 4.4

This occurs because the compiler inserts calls to the `deref_mut` method, to convert from `&mut AcquiredCown<Foo>` (which doesn't have fields `a` or `b`) to `&mut Foo` (which does). This happens both times `acq_cown` is dereferenced, with the compiler desugaring it into the code given in listing 4.6.

```
use_ints(
  &mut <AcquiredCown<Foo> as DerefMut>::deref_mut(&mut acq_cown).a,
  &mut <AcquiredCown<Foo> as DerefMut>::deref_mut(&mut acq_cown).b,
)
```

Listing 4.6: The desugaring of the call to `use_ints` in listing 4.4

This code has a compiler error (listing 4.7), as it attempts to borrow `acq_cown` mutably twice. Why then does listing 4.3 compile, when it seemingly does the same thing? The answer is that the borrow checker is able to understand that we are borrowing disjoint fields of a struct, and therefor the borrows are not overlapping (12).

However, when borrowing from an `AcquiredCown<T>` (instead of a `&mut T`), we must first borrow the *entire* cown to pass to the `deref_mut` method. Only then can we borrow the individual field that we want. At the time of the

deref_mut call, we have borrowed *all* of acq_cown. Whereas in listing 4.3, we never borrow all of foo, only its individual fields.

```
error[E0499]: cannot borrow `acq_cown` as mutable more than once at a time
--> tests/ui/partial-borrow.rs:30:65
|
28 | use_ints(
|   ----- first borrow later used by call
29 |     &mut <AcquiredCown<Foo> as DerefMut>::deref_mut(&mut acq_cown).a,
|                                                     ----- first mutable
|   ↪ borrow occurs here
30 |     &mut <AcquiredCown<Foo> as DerefMut>::deref_mut(&mut acq_cown).b,
|                                                     ^^^^^^^^^^^^^^^^^ second mutable
|   ↪ borrow occurs here
```

Listing 4.7: Compiler error from listing 4.6

It is possible to work around this. As shown in 4.8, one can instead first borrow the *entire* cown, and then borrow the fields on that.

```
when(&cown, |mut acq_cown: AcquiredCown<Foo>| {
    let mut_ref: &mut Foo = &mut *acq_cown;
    use_ints(&mut mut_ref.a, &mut mut_ref.b)
});
```

Listing 4.8: Version of 4.4 that compiles

Here, deref coercion only happens once, on the second line, to declare mut_ref. Then on the third line, we can split the borrow on mut_ref to get the two fields we want. This is allowed, because there's no hidden function calls here, so the borrow checker can locally check that we're obeying Aliasing XOR Mutation.

An alternative design that was considered was to have the closure in when be given &mut T (as opposed to AcquiredCown<T>). This would alleviate the borrowing issues discussed above. However, it would mean that code such as listing 4.9 would cease to work, as we'd no longer store the information that the argument was a cown anywhere in the system, so it wouldn't be possible to schedule another behaviour onto it.

```
when((a, b), |(a, b)| {
    do_something_with(a, b);
    when(&a.cown(), |a| do_something_else_with(a));
    when(&b.cown(), |b| do_a_third_thing_with(b));
});
```

Listing 4.9: Scheduling a new behaviour onto a cown acquired by another behaviour

4.2 Acquiring the Same Cown Twice

In Rust, it's undefined behaviour to have aliasing mutable references (13). Normally this is checked by the borrow checker (1.1.1.1), but when using unsafe as boxcars does, that code must ensure this is maintained.

However, when acquiring cowns, you could acquire the same cown twice, and then use those to obtain overlapping references, as shown in 4.10. While this is fine in BoC itself and doesn't violate any of its invariants or cause data-races for it, it's immediate undefined-behaviour in Rust.

To avoid this, boxcars will panic ¹ when attempting to schedule a behaviour onto the same cown twice. The output of this is shown in 4.11.

```
let c = Cown::new(10);
when((&c, &c), |(c1, c2)| {
    let ptr_1: &mut i32 = c1.deref_mut();
    let ptr_2: &mut i32 = c2.deref_mut();
    // aliasing mutable references, UB!!
});
```

Listing 4.10: Attempting to acquire the same cown twice, to obtain aliasing mutable references

```
thread 'main' panicked at crates/boxcars/src/when.rs:80:1:
Cowns not unique
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

Listing 4.11: Output of running listing 4.10

4.3 Compiler Errors

Both verona-rt and boxcars make extensive use of the type-system to ensure users code is correct. This means that when a user makes a mistake, this is often communicated to them in the form of a type error. In general, Rust (and boxcars by extension) is able to produce better type errors because it checks generics as soon as they're called, rather than instantiating templates and then checking them.

Note: Compiler errors are from g++ 13.2.0 and rustc 1.79.0. They will be slightly different on different versions.

4.3.1 Wrong Cown Constructor Arguments

The most frequent type error I ran into when writing the benchmarks was getting the wrong arguments to a constructor wrong. When the same mistake is made in both C++ (listing 4.12) and Rust (4.14), they both produce a compiler error (listings 4.13 and 4.15 respectively).

```
class Foo
{
    int number_;
    const char* str_;

public:
    Foo(int number, const char* str) : number_(number), str_(str) {}
};

//                                     These are the wrong way round
//                                     vvvvvvvvvvvvvv
auto c_foo = make_cown<Foo>("hello", 101);
```

Listing 4.12: Calling a constructor with the wrong arguments with verona_rt

¹To panic in Rust is to throw a non-recoverable error. It's like aborting, but with a bit more debugging machinery around it.

```

In file included from verona-rt/src/rt/./cpp/when.h:6,
                  from cpp/playground.cc:1:
verona-rt/src/rt/./cpp/cown.h: In instantiation of
  ↳ 'verona::cpp::ActualCown<T>::ActualCown(Args&& ...) [with Args = {const char
  ↳ (&)[6], int}; T = Foo]':
verona-rt/src/rt/./cpp/cown.h:374:24:   required from 'verona::cpp::cown_ptr<T>
  ↳ verona::cpp::make_cown(Args&& ...) [with T = Foo; Args = {const char (&)[6],
  ↳ int}]'
cpp/playground.cc:27:30:   required from here
verona-rt/src/rt/./cpp/cown.h:50:32: error: invalid conversion from 'const char*' to
  ↳ 'int' [-fpermissive]
   50 |         ActualCown(Args&&... ts) : value(std::forward<Args>(ts)...)
      |                                     ^~~~~~
      |                                     |
      |                                     const char*
cpp/playground.cc:21:11: note:   initializing argument 1 of 'Foo::Foo(int, const
  ↳ char*)'
   21 |     Foo(int number, const char* str) : number_(number), str_(str) {}
      |         ~~~~~^~~~~~
verona-rt/src/rt/./cpp/cown.h:50:32: error: invalid conversion from 'int' to 'const
  ↳ char*' [-fpermissive]
   50 |         ActualCown(Args&&... ts) : value(std::forward<Args>(ts)...)
      |                                     ^~~~~~
      |                                     |
      |                                     int
cpp/playground.cc:21:31: note:   initializing argument 2 of 'Foo::Foo(int, const
  ↳ char*)'
   21 |     Foo(int number, const char* str) : number_(number), str_(str) {}
      |

```

Listing 4.13: Compiler error from listing 4.12

```

struct Foo {
    number: i32,
    string: &'static str,
}

impl Foo {
    fn new(number: i32, string: &'static str) -> Self {
        Self { number, string }
    }
}

//                                     These are the wrong way round
//                                     vvvvvvvvvvvvvv
let c_foo = Cown::new(Foo::new("hello", 101));

```

Listing 4.14: Calling a constructor with the wrong arguments in boxcars


```
let a = Cown::<u32>::new(101);

// `a` should be of type `AcquiredCown<u32>`, not `AcquiredCown<bool>`
when(&a, |a: AcquiredCown<bool>| {});
```

Listing 4.18: Giving the wrong type for `AcquiredCown` in `boxcars`

[illegible]

4.3.3 Not Using a Tuple for When Lambda Arguments

This error's not applicable to verona-rt, but in boxcars

When acquiring multiple cowns, the argument to the lambda is a tuple of all the acquired cowns. However sometimes I typo'd (or forgot) this, and wrote out the arguments expecting to get all the cowns in different arguments, as shown in listing 4.19. Fortunately, because rustc knows exactly what type is expected at the call sight, it's able to produce a really great error (listing 4.20) that suggests how to fix this mistake.

```
let foo = Cown::new(101);
let bar = Cown::new(202);

when((&foo, &bar), |foo, bar| {});
```

Listing 4.19: boxars code that

4.3.4 Passing Wrong Arguments as Cowns

However there are some cases where `boxcars` can cause a quite unhelpful error to be emitted. For example, listing 4.21 passes a tuple of `cowns`, instead of a tuple of references to `cowns`. This causes the error given in listing 4.22. Instead of usefully describing the mistake and what is needed to fix it, it can only tell the user that the type is wrong.

```

error[E0593]: closure is expected to take a single 2-tuple as argument, but it takes
  ↳ 2 distinct arguments
  --> crates/verona-rt/examples/err.rs:22:5
22 |     when(&foo, &bar), |foo, bar| {});
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |     |                                     |
   |     |                                     takes 2 distinct arguments
   |     | expected closure that takes a single 2-tuple as argument
help: change the closure to accept a tuple instead of individual arguments
22 |     when(&foo, &bar), |(foo, bar)| {});
   |

```

Listing 4.20: Compiler error from listing 4.19

```

let foo = Cown::new(101);
let bar = Cown::new(202);

when((foo, bar), |(foo, bar)| {});

```

Listing 4.21: Not passing cowns by reference to when

The root of the problem is that you need to borrow these Cowns to form a CownCollection. However, because when's first argument is generic, rustc can't say what type it needs to be, only that it must implement a certain trait. This also causes a followup error, where it claims that the function has the wrong signature, because it can't find the correct one.

4.4 auto-copy vs explicit clone

In C++, the cown_ptr class overloads it's copy and assignment constructor to automatically update the reference count:

```
// class cown_ptr {
// Copy an existing cown ptr. Shares the underlying cown.
cown_ptr(const cown_ptr& other)
{
    allocated_cown = other.allocated_cown;
    if (allocated_cown != nullptr)
        verona::rt::Cown::acquire(allocated_cown);
}

// Copy an existing cown ptr. Shares the underlying cown.
cown_ptr& operator=(const cown_ptr& other)
{
    clear();
    allocated_cown = other.allocated_cown;
    if (allocated_cown != nullptr)
        verona::rt::Cown::acquire(allocated_cown);
    return *this;
}
```

This means that the following code Just Works:

```
auto c1 = make_cown<int>(10);
auto c2 = c1; // copy constructor
c2 = c1; // copy-assignment operator
```

And all the = magically update the reference count to be correct.

Whereas in Rust, because = moves (and there's no way to overload it), you need to instead write:

```
let c1 = Cown::new(10);
let mut c2 = c1.clone()
c2 = c1;
```

This adds visual clutter but also means that it's clearer when you're paying to cost to do reference counting.

```

error[E0277]: the trait bound `

```

Chapter 5

Future Works

In this project, I have successfully implemented a library that provides efficient access to BoC primitives in Rust. It invites a number of potential pieces of follow-up work:

5.1 Extending BoC and Boxcars

While `boxcars` implements all the behaviour described in the paper introducing BoC (10), `verona-rt` is currently developing a number of extensions to the core BoC primitives, such as:

1. Atomic scheduling of multiple behaviours
2. Read only acquired cowns, that allow
3. Notifications

`boxcars` could be extended to also support these additional features. This

5.2 Passing Cowns between Rust and C++

While currently `verona-rt` allows creating cowns with a C++ type and using them in C++, and `boxcars` allows creating cowns with a Rust type and using them in Rust, there is no way to create a cown that can be used in both languages. It would be nice to allow passing cowns across the FFI barrier between these languages. This would require changing the current design, where the C++ side is entirely unaware of the data stored in a `boxcars` cown.

5.3 Developing medium-scale software in BoC

Currently BoC has been used for small and microbenchmarks, but these have only been to demonstrate the features of BoC, and the performance of various implementations. Either `boxcars` or `verona-rt` could be used to build a piece of concurrent software that actual does something useful. With `boxcars` in particular, I'd be interested in finding out if the limitations on acquiring the same cown twice (§4.2) actually come up in practice.

References

1. Klabnik S, Nichols C. The rust programming language [Internet]. USA: No Starch Press; 2018 [cited 2024 Jun 11]. Available from: <https://doc.rust-lang.org/1.79.0/book/>
2. Köppe T, editor. Programming languages — c++. ISO/IEC JTC1 SC22 WG21; ISO, Geneva, Switzerland; 2020. Report No.: ISO/IEC 14882:2020.
3. Stroustrup B, Sutter H, editors. C++ core guidelines [Internet]. 2024 [cited 2024 Jun 16]. Available from: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>
4. Withoutboats S. Notes on a smaller rust [Internet]. 2019 [cited 2024 Jan 23]. Available from: <https://withoutboats.blog/notes-on-a-smaller-rust/>
5. Jung R. *Understanding and evolving the rust programming language*. 2020.
6. Tolnay D. Accurate mental model for rust’s reference types [Internet]. 2019 [cited 2024 Jan 23]. Available from: https://docs.rs/dtolnay/0.0.11/dtolnay/macro_02__reference_types.html
7. Snoyman M. My best and worst deadlock in rust [Internet]. 2024 [cited 2024 Jan 23]. Available from: <https://www.snoyman.com/blog/2024/01/best-worst-deadlock-rust/>
8. Wenger A. A rust match made in hell [Internet]. 2022 [cited 2024 Jan 23]. Available from: <https://fasterthanli.me/articles/a-rust-match-made-in-hell>
9. d’Antras A, Färnstrand L, Karan C, Kellum D, Noa, Nunley J, et al. `Parking_lot` [Internet]. 2024 [cited 2024 Jun 16]. Available from: https://github.com/Amanieu/parking_lot
10. Cheeseman L, Parkinson MJ, Clebsch S, Kogias M, Drossopoulou S, Chisnall D, et al. When concurrency matters: Behaviour-oriented concurrency. *Proc ACM Program Lang* [Internet]. 2023 Oct;7(OOPSLA2). Available from: <https://doi.org/10.1145/3622852>
11. Huss E, Scheel R, Klabnik S, Jasper M, Jung R, Woodcraft M, et al. The rust language reference [Internet]. [cited 2024 Jun 16]. Available from: <https://doc.rust-lang.org/1.79.0/reference/>
12. Beingessner A, Okushi Y, Huss E, Jung R, Klabnik S, Farrokhzad M, et al. The rustonomicon: The dark arts of advanced and unsafe rust programming [Internet]. [cited 2024 Jun 11]. Available from: <https://doc.rust-lang.org/1.79.0/nomicon/>
13. Jung R, Jourdan JH, Krebbers R, Dreyer D. RustBelt: Securing the foundations of the rust programming language. *Proc ACM Program Lang* [Internet]. 2017 Dec;2(POPL). Available from: <https://doi.org/10.1145/3158154>